01 · FALA PRINCIPAL · APROXIMADAMENTE 3 MINUTOS

Tecnologias e funcionamento do projeto

Roteiro em primeira pessoa, preparado para explicar a solução com clareza e precisão.

“O SkySync é um protótipo de apoio ao replanejamento de escalas de tripulação. O objetivo é acompanhar uma operação simulada e reagir quando uma designação deixa de ser viável, considerando a posição dos profissionais, suas horas acumuladas e as restrições implementadas.

Organizamos o sistema em três camadas: **interface, processamento e armazenamento**. Essa separação facilita a manutenção e permite testar as regras sem depender da apresentação visual.

Na interface, utilizamos **HTML, CSS e JavaScript**. O HTML estrutura as telas, o CSS cuida da aparência e da adaptação ao tamanho da tela, e o JavaScript consulta os dados e atualiza o painel, o mapa e as listas de ocorrências. Essa interface está hospedada no **GitHub Pages**.

O processamento utiliza **Python com Flask**, hospedado no **Render**. O Flask disponibiliza a API interna que conecta o navegador às funcionalidades do projeto. O backend executa a simulação, valida as condições de alocação e registra os resultados. Em produção, utilizamos o **Gunicorn** para servir a aplicação.

Para resolver a alocação, usamos o **Google OR-Tools, com o solver CP-SAT**. Ele trabalha com variáveis de decisão e restrições matemáticas. Na prática, o modelo decide quais profissionais podem ocupar cada função no voo. Na simulação, preservamos as designações que continuam válidas e buscamos substitutos quando necessário, sem deslocar artificialmente alguém de outro aeroporto.

O armazenamento utiliza **PostgreSQL, hospedado no Neon**. O Render faz os cálculos e o Neon guarda os cadastros, o progresso da simulação e o histórico das disrupções. Dessa forma, atualizar a página ou pausar a operação não apaga o que já aconteceu.

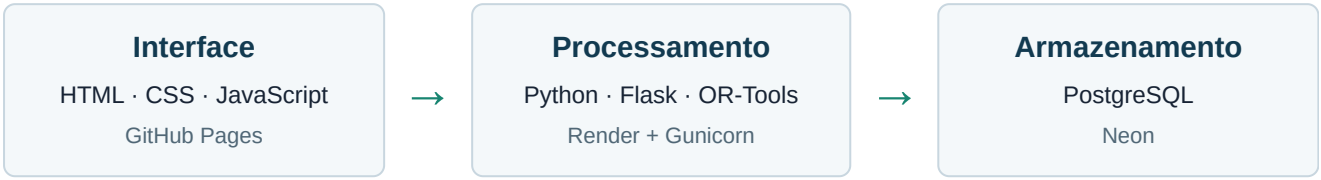
O mapa é desenhado em **SVG**, com um contorno geográfico local e coordenadas dos aeroportos. Ele acompanha o mesmo estado usado pelas jornadas e pelas disrupções. As posições dos aviões são calculadas a partir do tempo do voo simulado; não representam tráfego aéreo real.

Para validação, usamos **pytest** nos testes do backend e **Playwright** nos testes de navegador. O código é versionado com **Git e GitHub**. O principal resultado que demonstramos é a integração entre simulação, alocação sob restrições e registro dos motivos e resultados de cada alteração.”

02 · MATERIAL DE APOIO

O papel de cada tecnologia

Explique primeiro a função da camada e depois o nome da ferramenta.



O navegador troca dados JSON com a API interna. O backend acessa o banco por meio do driver psycopg e devolve os resultados para a interface.

Tecnologia	Como explicar à banca
HTML, CSS e JavaScript	Estruturam, apresentam e atualizam a interface. O projeto utiliza JavaScript sem framework de interface.
Python e Flask	Concentram a lógica da aplicação e expõem rotas HTTP para autenticação, operação, tripulantes e disrupções. Flask permite organizar essas rotas em módulos.
Google OR-Tools / CP-SAT	Resolve o modelo de alocação sob restrições. CP-SAT combina técnicas de programação por restrições e resolução de problemas de satisfatibilidade.
PostgreSQL e psycopg	PostgreSQL armazena os dados; psycopg conecta o código Python ao banco. Transações permitem salvar o avanço da operação e os eventos correspondentes de forma conjunta.
Neon	Hospeda o PostgreSQL na nuvem. É o local de armazenamento, não o serviço que executa o otimizador.
Render e Gunicorn	Render hospeda e executa o backend. Gunicorn atende às requisições da aplicação Flask em produção.
SVG e Natural Earth	SVG desenha o mapa no navegador. O contorno local do Brasil vem do Natural Earth; aeroportos e fundo usam a mesma projeção.
Git, GitHub e Pages	Git registra versões; GitHub hospeda o repositório e permite revisar alterações; GitHub Pages publica os arquivos da interface.
pytest e Playwright	pytest verifica regras e rotas do backend. Playwright exercita a interface em um navegador, incluindo navegação, pausa e atualização dos resultados.

> **Frase para memorizar:** “O navegador apresenta, o Render processa e o Neon armazena.”

> **API interna não é serviço externo de dados.** O SkySync continua usando sua própria API para funcionar. Os conectores externos de rastreamento e importação de voos foram removidos da versão atual.

03 · DO DADO À DECISÃO

Um exemplo para explicar o remanejamento

Exemplo ilustrativo. Os números abaixo servem para explicar a lógica; não são uma medição de resultado do projeto.

> “Imagine um voo saindo de Guarulhos para Manaus. O comandante previsto já tem **8 horas acumuladas**. A duração planejada é de **2 horas**, e o limite configurado para ele, neste exemplo, é de **11 horas**.”

1. **O plano inicial é viável.** O total projetado é de 10 horas. O profissional também precisa estar disponível, apto e no aeroporto de origem.
2. **A simulação introduz um atraso.** Um acréscimo de 90 minutos à duração faria o total chegar a 11 horas e 30 minutos, ultrapassando o limite configurado.
3. **A designação é reavaliada.** O sistema mantém os demais profissionais que continuam aptos e procura um comandante substituto em Guarulhos, com capacidade de horas e condições para executar o voo.
4. **O otimizador busca uma composição viável.** Se encontrar um substituto, a troca é registrada com quem saiu, quem entrou e o motivo. Se não encontrar, o voo fica aguardando tripulação na origem.
5. **O voo e as horas avançam juntos.** Após a decolagem simulada, o avião percorre o trajeto no mapa e os profissionais alocados acumulam minutos de voo. A localização dos tripulantes muda no pouso.
6. **As disrupções acompanham o resultado.** O remanejamento fica no histórico. A falta de tripulação é encerrada quando o voo decola, e o atraso é encerrado quando o voo pousa.

Como mostrar isso durante a apresentação

Abra **Escalas** para iniciar ou continuar a operação. No **Painel**, selecione um voo para mostrar sua rota e tripulação. Em **Disrupções**, apresente o motivo e o resultado de uma ocorrência. Por fim, pause a operação para demonstrar que o relógio e o mapa param juntos.

> **Explique o alcance da simulação:** o relógio avança 60 vezes mais rápido; as horas aumentam para os profissionais em voo. O protótipo usa simplificações de intervalo e repouso, incluindo 30 minutos em solo e 12 horas de repouso. Esses parâmetros não representam, sozinhos, toda a legislação de jornada.

Sequência para lembrar:

Estado da operação → identificação do impedimento → busca de substituto apto → registro da decisão → atualização do mapa e do histórico.

04 · RESPOSTAS CURTAS E LIMITES ATUAIS

Como responder com segurança

Por que vocês escolheram Python e Flask?

“Python permite integrar as regras e o OR-Tools. O Flask oferece uma estrutura direta para construir a API e separar as funcionalidades, adequada ao escopo do protótipo.”

O sistema usa inteligência artificial?

“Usamos otimização combinatória por restrições. Não treinamos um modelo com exemplos nem usamos IA generativa para decidir as escalas; as decisões dependem do modelo matemático e dos dados de entrada.”

O que significa uma variável de decisão?

“Para um par de tripulante e voo, uma variável binária indica se ele será alocado ou não. As restrições limitam as combinações permitidas, como composição por cargo e capacidade de horas.”

O sistema sempre encontra a melhor solução?

“Não necessariamente. O solver pode encontrar uma solução viável sem provar que ela é ótima dentro do tempo disponível. Se não houver solução utilizável, a simulação mantém o voo aguardando tripulação.”

Qual é a função objetivo e ela equilibra a carga?

“O código atual minimiza a soma das folgas de jornada. Com voos e composição fixos, essa soma pode permanecer constante entre alocações. Por isso, não afirmamos que ela comprova balanceamento; um objetivo que diferencie melhor as soluções é um ponto de evolução.”

Os dados e os voos são reais?

“As tripulações e a operação apresentada são simuladas. Usamos referências geográficas para os aeroportos e o mapa. O objetivo é construir cenários controlados e observar a reação do sistema.”

O projeto garante toda a conformidade com a RBAC 117?

“O protótipo implementa um conjunto de restrições com referência à regulamentação e usa simplificações na simulação. Não equivale a uma validação completa para uso operacional em uma companhia aérea.”

Onde os resultados ficam salvos?

“No PostgreSQL hospedado no Neon. O Render executa os cálculos. O progresso e as ocorrências são persistidos; no plano gratuito, o avanço da simulação para enquanto o servidor está suspenso.”

Como vocês verificaram o funcionamento?

“Com testes das regras e da API, testes de navegador e verificações com PostgreSQL. Avaliamos acúmulo de horas, localização, pausa, remanejamentos, persistência e ausência de duplicação dos eventos.”

> **Antes de apresentar:** abra o site, confira o acesso e escolha um voo ou ocorrência para demonstrar. Use resultados observados no teste de vocês e diferencie o que já está implementado do que será avaliado nas próximas etapas.